

Introduction

Programming for Economists

Jeppe Druedahl and Brigitte Hochmuth

Autumn 2026

1. Intended learning goals
2. Course plan
3. Exam
4. Active learning
5. Programming in practice
6. Summing up

Intended learning goals

Intended learning goals

- **In a nutshell:**

1. Summarize and visualize economic data
 - build upon Descriptive Economics
 - gain flexibility and automation compared to Excel
 - work with larger and messier data sets
2. Numerically solve and simulate economic models
 - use fewer simplifying assumptions than on paper
 - connect programming to Micro and Macro
 - move from model-user to *model-builder*

Intended learning goals

- **In a nutshell:**

1. Summarize and visualize economic data

- build upon Descriptive Economics
- gain flexibility and automation compared to Excel
- work with larger and messier data sets

2. Numerically solve and simulate economic models

- use fewer simplifying assumptions than on paper
- connect programming to Micro and Macro
- move from model-user to *model-builder*

- **Tool:** *Python*, a general-purpose programming language (open source)

1. Web development (not covered)

2. Data science and machine learning (covered as tools)

3. Scientific computing and research (main focus)

Why programming matters

- Economic data rarely arrive as clean Excel files
 - you often need to download, clean, merge and document data
- Programming makes work reproducible
 - the same code can update a figure when new data arrive
- Models become easier to explore
 - change a parameter, re-solve the model, plot the result
- These skills matter for later courses, theses, internships and policy work

Why learn this when AI can help?

- AI can help with syntax, debugging and first drafts of code
- AI does not know whether your data definition or model assumption is right
- You must be able to read, test and defend the code you submit
- In some settings, for example confidential data servers, AI tools may not be allowed
- The goal is not to memorize syntax. The goal is to understand the workflow.

Course plan

Lectures:

- **2–5 – Basics:** Fundamental concepts and NumPy
- **6–8 – Tools:** Optimization, root-finding, random numbers and simulation
- **9–10 – Data:** Load/save, use online APIs, clean, structure and analyze data
- **11–12 – Models:** Solow (macro) and Walras (micro)

Classes:

- **1:** Installation of Python and VSCode
- **1–4:** Online DataCamp + small problem sets
- **5–14:** Full problem sets and the data and model projects

September 9

- There is no in-person lecture on Wednesday, September 9
- Use the week for structured self-study
 1. make sure Python, Anaconda and VSCode work
 2. work on Problem Set 0
 3. start the assigned DataCamp chapters
- Lecture on September 16 begins with a short recap and coding exercises

- **Web-site:** ProgEcon course website
 1. installation guides and VSCode guide
 2. calendar and practical information
- **Course material in two GitHub repositories**
 1. ProgEcon-lectures: slides, notebooks, course plan and projects
 2. ProgEcon-exercises: problem sets and solutions
- **Need-to-know:** Click on »<> Code« and »Download zip«

Exam

Exam and projects

- **Exam requirements** (see [deadlines](#))
 1. Required DataCamp courses ([DataCamp.com](#))
 2. Project 1: Data project
 3. Project 2: Model project
- **Exam:** Portfolio of revised projects (40%)
+ 36 hours home assignment (60%)
- **Grading:** Pass or fail
- **Groups:** All projects can be done in fixed groups (maximum of 4)
 - Groups can include students from different holds
 - Sign up your group in the [Google Sheet](#)

DataCamp and early problem sets

- Access DataCamp here: [DataCamp course link](#)
- You must complete the following DataCamp courses
 1. Introduction to Python
 2. Intermediate Python
- Optional extra practice
 1. Introduction to Functions in Python
 2. Introduction to NumPy
 3. Intermediate Object-Oriented Programming in Python
- Early exercise classes combine DataCamp with small accompanying problem sets

DataCamp deadlines

All deadlines are Sundays at 23:59.

Deadline	Required progress
----------	-------------------

Sep 13	Introduction to Python
--------	-------------------------------

	Ch. 1: <i>Python Basics</i>
--	-----------------------------

	Ch. 2: <i>Python Lists</i>
--	----------------------------

Sep 20	Introduction to Python
--------	-------------------------------

	Ch. 3: <i>Functions and Packages</i>
--	--------------------------------------

	Ch. 4: <i>NumPy</i> Course completed
--	---

Sep 27	Intermediate Python
--------	----------------------------

	Ch. 1: <i>Matplotlib</i>
--	--------------------------

	Ch. 2: <i>Dictionaries & Pandas</i>
--	---

Oct 4	Intermediate Python
-------	----------------------------

	Ch. 3: <i>Logic, Control Flow and Filtering</i>
--	---

	Ch. 4: <i>Loops</i>
--	---------------------

Oct 11	Intermediate Python
--------	----------------------------

	Ch. 5: <i>Case Study: Hacker Statistics</i> Course completed
--	---

- **Objectives:**

1. Import and work with data
2. Apply data cleaning and structuring methods
3. Apply data analysis methods
4. Structure a code project
5. Document code
6. Present results in text and figures

- **Content:**

1. Import data from an online source
2. Clean and structure the data
3. Present and visualize the data
4. Apply methods from Descriptive Economics

- **Structure:**

1. A self-contained notebook presenting the analysis
2. Fully documented project files

- **Hand-in:** Upload via **Dropbox** by **Sunday, November 29, 23:59**

- **Objectives:**

1. Apply model analysis methods
2. Structure a code project
3. Document code
4. Present results in text and figures

- **Content:**

1. Describe an algorithm for solving a simple economic model
2. Solve or simulate the model
3. Visualize results across parameter choices
4. Analyze one or more extensions of the baseline model

- **Structure:**

1. A self-contained notebook presenting the analysis
2. Fully documented project files

- **Hand-in:** Upload via [Dropbox](#) by **Sunday, December 13, 23:59**

Active learning

- **Active learning:** To learn programming you need to work on actual problems yourself
 - We give you a broad overview
 - We give you a set of important tools
 - You have to experiment on your own
 - You have to be able to acquire new tools on your own for further studies and work

- **Active learning:** To learn programming you need to work on actual problems yourself
 - We give you a broad overview
 - We give you a set of important tools
 - You have to experiment on your own
 - You have to be able to acquire new tools on your own for further studies and work
- **Use Copilot and LLMs such as ChatGPT**
 - From words to code
 - Less need to remember syntax
 - Useful for explaining errors and suggesting solutions
 - Answers are often easy to verify, if you understand what the code does

- **Active learning:** To learn programming you need to work on actual problems yourself
 - We give you a broad overview
 - We give you a set of important tools
 - You have to experiment on your own
 - You have to be able to acquire new tools on your own for further studies and work
- **Use Copilot and LLMs such as ChatGPT**
 - From words to code
 - Less need to remember syntax
 - Useful for explaining errors and suggesting solutions
 - Answers are often easy to verify, if you understand what the code does
- **High level:** The goal is not just to learn syntax, but to become able to work independently with data and economic models

- Numerical analysis is a *complement*, not a substitute for math

- Numerical analysis is a *complement*, not a substitute for math
- Three central steps:
 1. mathematical problem → construct algorithm
 2. algorithm → write code
 3. run code → present results

- **Numerical analysis** is a *complement*, not a substitute for math
- **Three central steps:**
 1. mathematical problem → construct algorithm
 2. algorithm → write code
 3. run code → present results
- **The set of potential errors is very large:**

A good work-flow is therefore important

 1. Clear structure reduces the number of bugs
 2. Testing helps discover bugs
 3. Documentation helps remove bugs

- **Numerical analysis** is a *complement*, not a substitute for math
- **Three central steps:**
 1. mathematical problem → construct algorithm
 2. algorithm → write code
 3. run code → present results
- **The set of potential errors is very large:**

A good work-flow is therefore important

 1. Clear structure reduces the number of bugs
 2. Testing helps discover bugs
 3. Documentation helps remove bugs
- **Programming is more than writing code:**

Structuring, testing, documenting and collaborating on code projects are central parts of this course

Learning phases

1. **Basics:** A million new concepts and syntax.
How does it all fit together?!?!?

Learning phases

1. **Basics:** A million new concepts and syntax.
How does it all fit together?!?!?
2. **Tools:** Optimization, root-finding and simulation can feel abstract at first.
But you can start solving problems that are difficult to solve by hand.

Learning phases

1. **Basics:** A million new concepts and syntax.
How does it all fit together?!?!?
2. **Tools:** Optimization, root-finding and simulation can feel abstract at first.
But you can start solving problems that are difficult to solve by hand.
3. **Data:** It is also hard to use in practice.
But I begin to be able to do some cool stuff.

Learning phases

1. **Basics:** A million new concepts and syntax.
How does it all fit together?!?!?
2. **Tools:** Optimization, root-finding and simulation can feel abstract at first.
But you can start solving problems that are difficult to solve by hand.
3. **Data:** It is also hard to use in practice.
But I begin to be able to do some cool stuff.
4. **Models:** Hard to understand math and code simultaneously.
But the combination is really powerful.

- **Lectures:**

1. Attend lectures and *participate actively*
2. *Try out the code yourself*

Your work-flow

- **Lectures:**

1. Attend lectures and *participate actively*
2. *Try out the code yourself*

- **Exercises:** Short introduction + you work on problem sets, DataCamp or projects

1. Solve small tasks
2. Fill in missing code
3. Find errors
4. Solve full problems
5. Ask questions

Note: OK to peek at answers, *but write the solution yourself*

Your work-flow

- **Lectures:**

1. Attend lectures and *participate actively*
2. *Try out the code yourself*

- **Exercises:** Short introduction + you work on problem sets, DataCamp or projects

1. Solve small tasks
2. Fill in missing code
3. Find errors
4. Solve full problems
5. Ask questions

Note: OK to peek at answers, *but write the solution yourself*

- **In between classes and lectures:**

1. Go through lecture notebooks
2. Solve the problem set
3. Experiment with your own ideas

- **Observation:** Programming often means removing many small errors in your code, one at a time

- **Observation:** Programming often means removing many small errors in your code, one at a time
- **Everybody forgets the correct syntax**
⇒ trial-and-error and testing are central parts of programming

- **Observation:** Programming often means removing many small errors in your code, one at a time
- **Everybody forgets the correct syntax**
⇒ trial-and-error and testing are central parts of programming
- **Many sources for answers:**
 1. Documentation and course material
 2. Copilot and LLMs
 3. Fellow students
 4. Teaching assistants and teachers

- **Observation:** Programming often means removing many small errors in your code, one at a time
- **Everybody forgets the correct syntax**
⇒ trial-and-error and testing are central parts of programming
- **Many sources for answers:**
 1. Documentation and course material
 2. Copilot and LLMs
 3. Fellow students
 4. Teaching assistants and teachers
- **Help each other!** You will learn a lot from explaining code to somebody else
 - Keep feedback constructive

Programming in practice

Python distribution and VSCode

- We work with **Python 3.x**
- **Suggested environment:**
 1. **Distribution:** Anaconda
 2. **Editor/IDE:** VSCode
 3. **Notebooks:** Jupyter notebooks inside VSCode
- **Examples today:**
 1. Run Python code and notebooks
 2. Load some economic data and make a plot
 3. Change a parameter and see how the outcome changes
- **The point today:** You do not need to understand every detail. You should get an idea of what you will be able to do by the end of the course.

Summing up

Summing up

- **I hope you now have an idea of:**
 1. Why learning programming and numerical analysis is important
 2. What you will learn in this course
 3. Why active participation is required
 4. How DataCamp, classes, projects and the exam fit together

Your to-do list

1. **First priority:** Log in to [DataCamp.com](https://datacamp.com) (see information in the e-mail)

Your to-do list

1. **First priority:** Log in to DataCamp.com (see information in the e-mail)
2. **Second priority:** Install **Anaconda** and **VSCode** (see the [installation guide](#))

Your to-do list

1. **First priority:** Log in to [DataCamp.com](https://datacamp.com) (see information in the e-mail)
2. **Second priority:** Install **Anaconda** and **VSCode** (see the [installation guide](#))
3. **Third priority:** Download the lecture and exercise material

Your to-do list

1. **First priority:** Log in to DataCamp.com (see information in the e-mail)
2. **Second priority:** Install **Anaconda** and **VSCode** (see the [installation guide](#))
3. **Third priority:** Download the lecture and exercise material
4. **Fourth priority:** Open the introductory notebook and run the example code yourself

Your to-do list

1. **First priority:** Log in to DataCamp.com (see information in the e-mail)
2. **Second priority:** Install **Anaconda** and **VSCode** (see the [installation guide](#))
3. **Third priority:** Download the lecture and exercise material
4. **Fourth priority:** Open the introductory notebook and run the example code yourself
5. **Fifth priority:** Complete Chapters 1–2 of *Introduction to Python* by **Sunday, September 13, 23:59**